# CS 300 Project One: Runtime and Memory Analysis

**Reem Lewis | Southern New Hampshire University | CS-300**

ABCU Advising Program — comparing a vector, a hash table, and a binary search tree for storing and retrieving Computer Science course data.

## Runtime and Memory Evaluation

I analyzed the worst-case running time of reading the file and creating course objects for each structure, where n is the number of courses and c is the small, constant maximum number of prerequisites per course. Each line costs 1 unless it calls a function, in which case it costs that function's running time. In the worst case all three structures load in $O(n^2)$: the vector scans every course for each duplicate and prerequisite check, the hash table becomes one long chain if every course number collides, and the tree becomes a chain if courses arrive already sorted. On typical data they separate: the vector stays $O(n^2)$, the hash table loads in $O(n)$, and the tree in $O(n \log n)$. Memory is $O(n)$ for all three. The vector stores courses back to back with no extra links, the hash table adds a bucket array and one link per course, and the tree adds two child links per course.

### Summary of Results

| Requirement | Vector | Hash table | Binary search tree |
|---|---|---|---|
| Load file (worst case) | $O(n^2)$ | $O(n^2)$ | $O(n^2)$ |
| Load file (average case) | $O(n^2)$ | $O(n)$ | $O(n \log n)$ |
| Print sorted list | $O(n \log n)$ average sort | $O(n \log n)$: copy, then sort | $O(n)$: in-order traversal |
| Find one course | $O(n)$ | $O(1)$ average, $O(n)$ worst | $O(\log n)$ average, $O(n)$ worst |
| Memory | $O(n)$, no extra links | $O(n + m)$: m buckets plus one link per course | $O(n)$: two child links per course |

## Advantages and Disadvantages

**Vector:** It is the simplest structure and uses the least memory. Its drawbacks are that every search is O(n), including each prerequisite lookup, so loading is $O(n^2)$, and the courses must be sorted with quicksort (O(n log n) on average) every time the list is printed.

**Hash table:** Looking up a course by course number is O(1) on average, which makes loading and prerequisite checks the fastest. However, a hash table has no order, so printing alphanumerically means copying every course into a vector and sorting it. Collisions can degrade it to O(n), and the bucket array uses extra space.

**Binary search tree:** It keeps courses sorted by course number, so printing the course list is a plain in-order traversal in O(n) with no sorting, and lookups are O(log n) on average. Its drawbacks are the extra pointer memory and more complex code. If courses are inserted in order, the tree becomes a chain and operations become O(n).

## Recommendation

I recommend the binary search tree. The advisors need two things: an alphanumeric list of every course and the title and prerequisites of one course. The tree is the only structure that does both well without extra work. Its in-order traversal prints the sorted list in O(n), while the vector and hash table must first sort in O(n log n), and its lookups average O(log n) compared with O(n) for the vector. The hash table finds one course faster, but that gain is small for a course catalog of ABCU's size and it gives up sorted order. The tree's main risk is height: the sample file is mostly in order, so its tree is a chain with 8 levels for 8 courses. That is harmless at this size, and if the catalog grows I would use a self-balancing tree to keep lookups at O(log n) in the worst case.